

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

13 - Logical Operations

Introduction

- Logical operations

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit NOT	~	~	nor

Shift Operations

- Move all bits in word to left or right, filling emptied bits with 0s

Shift Operations

- Move all bits in word to left or right, filling emptied bits with 0s
- For example, if register \$s0 contained

0000 0000 0000 0000 0000 0000 0000 1001_{two} = 9_{ten}

instruction to shift left by 4 was executed, new value would be:

Shift Operations

- Move all bits in word to left or right, filling emptied bits with 0s
- For example, if register \$s0 contains

0000 0000 0000 0000 0000 0000 0000 1001_{two} = 9_{ten}

instruction to shift left by 4 is executed, new value would be:

0000 0000 0000 0000 0000 0000 1001 0000_{two} = 144_{ten}

Shift Instructions

- Dual of **shift left** is **shift right**
- Actual name of the two MIPS shift instructions are called
 - *shift left logical* (**sll**)
 - *shift right logical* (**srl**)

Shift Instructions

- The following instruction performs shift operation, assuming that original value was in register \$s0 and result should go in register \$t2:

`sll $t2,$s0,4 # reg $t2 = reg $s0 << 4 bits`

- *shamt* field in R-format is used in shift instructions
 - Stands for *shift amount*

MIPS Registers

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

Shift Instructions

- Machine language version of the instruction is:

op	rs	rt	rd	shamt	funct
0	0	16	10	4	0

- Encoding of `sll`
 - 0 in `op` and `funct` fields
 - `rd` contains 10 (register \$t2), `rt` contains 16 (register \$s0)
 - `shamt` contains 4
 - `rs` field is unused and thus is set to 0

Benefits

- Shifting left by i bits gives same result as multiplying by 2^i
- For example, previous s11 shifts by 4, which gives same result as multiplying by 2^4 or 16
 - First bit pattern is 9, and $9 \times 16 = 144$, value of second bit pattern
- Shifting right by i bits gives same result as dividing by 2^i

AND Operation

- Bit-by-bit operation that leaves 1 in result only if both bits of operands are 1
- MIPS Instruction: `and`

Example

- After executing the MIPS instruction:

`and $t0,$t1,$t2` # reg \$t0 = reg \$t1 & reg \$t2

\$t2 0000 0000 0000 0000 0000 1101 1100 0000_{two}

\$t1 0000 0000 0000 0000 0011 1100 0000 0000_{two}

value of register \$t0 would be

\$t0 0000 0000 0000 0000 0000 1100 0000 0000_{two}

OR Operation

- Bit-by-bit operation that places 1 in result if either operand bit is 1
- MIPS instruction: **or**

Example

- After executing the MIPS instruction:

`or $t0,$t1,$t2` # reg \$t0 = reg \$t1 | reg \$t2

\$t2 0000 0000 0000 0000 0000 1101 1100 0000_{two}

\$t1 0000 0000 0000 0000 0011 1100 0000 0000_{two}

value of register \$t0 would be

\$t0 0000 0000 0000 0000 0011 1101 1100 0000_{two}

NOT Operation

- Takes one operand and places 1 in result if one operand bit is 0, and vice versa
- In keeping with three-operand format, designers of MIPS decided to include instruction **NOR (NOT OR)** instead of NOT
- If one operand is zero, then it is equivalent to NOT
 - $A \text{ NOR } 0 = \text{NOT } (A \text{ OR } 0) = \text{NOT } (A)$
- MIPS instruction: **nor**

Example

- If register \$t1 contains the value

\$t1 0000 0000 0000 0000 0011 1100 0000 0000

and register \$t3 has the value 0

result of the MIPS instruction

`nor $t0, $t1, $t3` # reg \$t0 = $\sim$ (reg \$t1 | reg \$t3)

would be

\$t0 1111 1111 1111 1111 1100 0011 1111 1111

NOT Operations

- Useful to invert bits in a word
- MIPS has NOR 3-operand instruction
 - $a \text{ NOR } b == \text{NOT} (a \text{ OR } b)$

```
nor $t0, $t1, $t2
```

```
nor $t0, $t1, $zero
```

\$t1

0000 0000 0000 0000 0011 1100 0000 0000
